

DATA STRUCTURES LAB

Elementary Sorting Algorithms



Foundational Sorting Concepts

Lecture Overview

Stability

A sorting algorithm is **Stable** if it preserves the relative order of equal key values present in the input array.

Example: $[5a, 3, 5b] \rightarrow [3, 5a, 5b]$

In-Place Sorting

An algorithm is **In-Place** if it requires only a constant amount of additional memory space to perform the sort.

No extra dynamic arrays created

Swaps vs Shifts

Swaps exchange two elements using temporary variable. **Shifts** move elements rightward to create space for an insertion.

Key metric for performance

ALGORITHM 01

Bubble Sort

Repeatedly comparing adjacent elements and swapping them if they are in wrong order until the entire array is sorted.

Bubble Sort: Core Mechanics

Concept

💡 How It Works

- Compares adjacent elements `arr[j]` and `arr[j + 1]` starting from index 0.
- If `arr[j] > arr[j + 1]`, swap them to place larger element to the right.
- In pass i , the i -th largest element "bubbles up" to its correct position at the end.
- For an array of size N , exactly $N-1$ passes are required.

Bubble Sort: Dry Run (Pass 1 & 2)

[Trace Example](#)

Input Array (6 numbers): [5, 2, 8, 1, 9, 3]

Pass 1 (i = 0)

Compare 5 & 2 → Swap



Compare 5 & 8 → No Swap

5 < 8, array remains [2, 5, 8, 1, 9, 3]

Compare 8 & 1 → Swap | 8 & 9 → No Swap

State: [2, 5, 1, 8, 9, 3]

Compare 9 & 3 → Swap



Result: 9 bubbled to index 5

Pass 2 (i = 1)

Compare 2 & 5 → No Swap

Array: [2, 5, 1, 8, 3, 9]

Compare 5 & 1 → Swap

Array becomes: [2, 1, 5, 8, 3, 9]

Compare 5 & 8 → No Swap

Array: [2, 1, 5, 8, 3, 9]

Compare 8 & 3 → Swap



Result: 8 bubbled to index 4

Bubble Sort: Dry Run (Passes 3, 4 & 5)

[Trace Example](#)

Pass 3 (i = 2)

Compare 2 & 1 → Swap

[1, 2, 5, 3, 8, 9]

Compare 2 & 5 → No Swap

[1, 2, 5, 3, 8, 9]

Compare 5 & 3 → Swap



Pass 4 (i = 3)

Compare 1 & 2 → No Swap

[1, 2, 3, 5, 8, 9]

Compare 2 & 3 → No Swap



Pass 5 (i = 4)

Compare 1 & 2 → No Swap

Final pass completes!

Final Sorted Array



Bubble Sort: Pseudocode & C++ Code

Implementation

Pseudocode

```
Algorithm BasicBubbleSort(A, N):
  For i = 0 To N - 2:
    For j = 0 To N - i - 2:
      If A[j] > A[j + 1] Then
        // Swap elements
        temp = A[j]
        A[j] = A[j + 1]
        A[j + 1] = temp
      EndIf
    EndFor
  EndFor
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void bubbleSort(int arr[], int n) {
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }
}
```

Bubble Sort: Complexity & Properties

Analysis

Time & Space Complexity

BEST TIME (BASIC)

$$O(N^2)$$

AVERAGE TIME

$$O(N^2)$$

WORST TIME

$$O(N^2)$$

AUXILIARY SPACE

$$O(1)$$

Key Characteristics

- **Stability:** **YES** (Does not swap equal elements).
- **In-Place:** **YES** (Modifies input array directly).
- **Comparisons:** Always $\frac{N(N-1)}{2}$ comparisons in basic version.
- **Swaps:** Worst case $\frac{N(N-1)}{2}$ swaps (Reverse sorted array).
- **Primary Use:** Educational tool to introduce sorting loops.

ALGORITHM 02

Selection Sort

Repeatedly finding the minimum element from the unsorted subarray
and placing it at the beginning.

Selection Sort: Core Mechanics

Concept

🎯 How It Works

- Divides array into **Sorted Subarray** (left) and **Unsorted Subarray** (right).
- Scans the unsorted part to find index of the **minimum element** (`minIndex`).
- Swaps the minimum element with the **first element of the unsorted part**.
- Expands the sorted boundary by 1 in each pass.

💎 Unique Feature: Minimum Swaps

Unlike Bubble Sort, Selection Sort performs at most $N - 1$ **swaps** in total across the entire algorithm execution.

When to Use:

Ideal when writing to memory is expensive or slow (e.g., Flash memory writes).

Selection Sort: Dry Run Step-by-Step

[Trace Example](#)

Input Array (6 elements): [64, 25, 12, 22, 11, 30]

Pass 1 (i = 0): Find Min in [64, 25, 12, 22, 11, 30]

Minimum is 11 at index 4 → Swap with arr[0] (64)



Pass 2 (i = 1): Find Min in [25, 12, 22, 64, 30]

Minimum is 12 at index 2 → Swap with arr[1] (25)



Pass 3 (i = 2): Find Min in [25, 22, 64, 30]

Minimum is 22 at index 3 → Swap with arr[2] (25)



Pass 4 & 5: Find Min in remaining elements

Pass 4: Min is 25 (No swap needed)

Pass 5: Min is 30 → Swap 30 with 64



Selection Sort: Pseudocode & C++ Code

Implementation

Pseudocode

```
Algorithm SelectionSort(A, N):
  For i = 0 To N - 2:
    minIdx = i
    For j = i + 1 To N - 1:
      If A[j] < A[minIdx] Then
        minIdx = j
      EndIf
    EndFor
    If minIdx ≠ i Then
      Swap(A[i], A[minIdx])
    EndIf
  EndFor
```

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(int arr[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int minIdx = i;
    for (int j = i + 1; j < n; j++) {
      if (arr[j] < arr[minIdx]) minIdx = j;
    }
    if (minIdx ≠ i) {
      int temp = arr[i];
      arr[i] = arr[minIdx];
      arr[minIdx] = temp;
    }
  }
}
```

Selection Sort: Complexity & Properties

Analysis

Performance Metrics

BEST TIME

$O(N^2)$

AVERAGE TIME

$O(N^2)$

WORST TIME

$O(N^2)$

AUX SPACE

$O(1)$

Characteristics & Caution

- **Stability:** **NO** (Default version is Unstable due to long swaps).
- **In-Place:** **YES** $O(1)$ (space).
- **Comparisons:** Always $\frac{N(N-1)}{2}$ across all cases.
- **Swaps:** At most $O(N)$ swaps (strictly $N-1$ swaps).
- **Non-Adaptive:** Performs same comparisons regardless of input order.

ALGORITHM 03

Insertion Sort

Building the sorted array one element at a time by inserting each item into its proper sorted position.

Insertion Sort: Core Mechanics

Concept

Card Player Analogy

Similar to how people sort playing cards in their hands:

- Assume first element is already sorted.
- Pick the next element (called the **key**).
- Compare **key** with elements in sorted subarray from right to left.
- Shift elements greater than key one position to the right.
- Insert the key into its correct slot.

⚡ Adaptive & Efficient

Insertion Sort is **Adaptive**! It runs in $O(N)$ **linear time** if the input array is already or nearly sorted.

Hybrid Algorithms:

Used inside `std::sort` (Timsort/Introsort) for small arrays ($N \leq 16$).

Insertion Sort: Dry Run Step-by-Step

[Trace Example](#)

Input Array (6 elements): [12, 11, 13, 5, 6, 7]

i = 1 (Key = 11): Compare key with 12

12 > 11 → Shift 12 right, insert key 11 at index 0



i = 2 (Key = 13): Compare key with 12

13 > 12 → No shift required. 13 stays at index 2



i = 3 (Key = 5): Compare key with 13, 12, 11

Shift 13, 12, 11 right → Insert 5 at index 0



i = 4 & 5 (Keys 6, 7): Shift and Insert

Insert 6 after 5; Insert 7 after 6



Insertion Sort: Pseudocode & C++ Code

Implementation

Pseudocode

```
Algorithm InsertionSort(A, N):
  For i = 1 To N - 1:
    key = A[i]
    j = i - 1
    // Shift elements > key
    While j ≥ 0 and A[j] > key:
      A[j + 1] = A[j]
      j = j - 1
    EndWhile
    A[j + 1] = key
  EndFor
```

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void insertionSort(int arr[], int n) {
  for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j ≥ 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j--;
    }
    arr[j + 1] = key;
  }
}
```

Algorithm Summary & Comparison

Lab Takeaways

Algorithm	Best Time	Avg Time	Worst Time	Space	Stable?	Key Advantage
Bubble Sort (Basic)	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Simple logic & easy adjacent swaps
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$	No	Minimum swaps: strictly $O(N)$ writes
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Fastest for small / partially sorted data